Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: D.Kanishkarane
Reg No: 24011101021

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Results and Discussion

Four weight initialization strategies were evaluated on the same network architecture and training configuration for 6 epochs each: Zero, Random, Xavier (Glorot), and He initialization. The training loss and validation accuracy curves for each strategy are shown below.

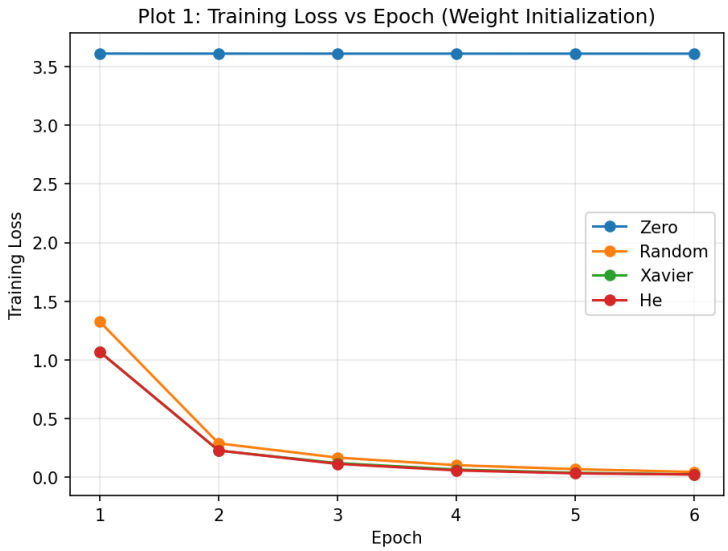


Figure 1: Training Loss vs. Epoch for different weight initialization strategies

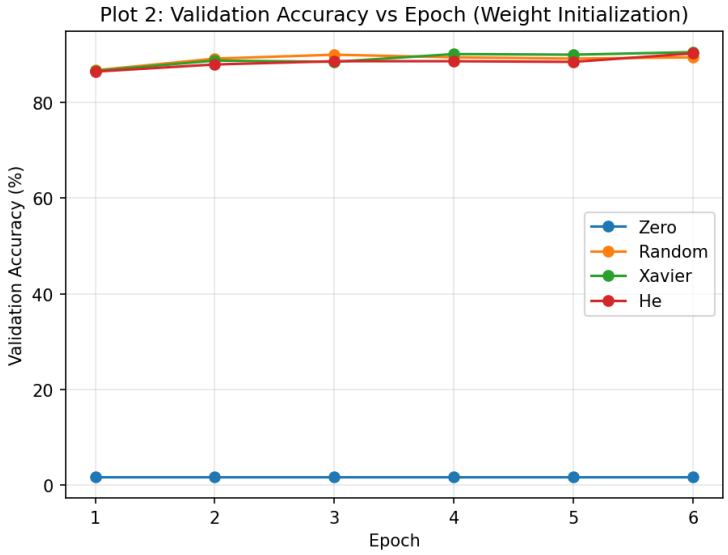


Figure 2: Validation Accuracy vs. Epoch for different weight initialization strategies

Table 1: Final epoch results for different weight initialization strategies

Initialization	**Train Acc. (%)**	**Train Loss**	**Val. Acc. (%)**	**Val. Loss**
Zero	2.96	3.6107	1.77	3.6141
Random	99.59	0.0435	89.40	0.2995
Xavier	99.90	0.0226	90.49	0.2879
He	99.80	0.0222	90.22	0.2825

Inference: Zero initialization completely fails to train the network. Since every neuron in a layer starts with identical weights, all neurons receive identical gradients during backpropagation and update identically throughout training, so the layer never breaks symmetry and behaves as a single effective neuron. This is reflected in the flat training loss of 3.6107 and validation accuracy stuck at 1.77%, essentially random guessing over the 37 pet classes.

Random initialization breaks this symmetry and enables learning, but without any variance scaling based on layer size, the initial activations and gradients are poorly conditioned, leading to slightly noisier convergence and the weakest validation accuracy (89.40%) among the three working strategies, along with a marginally higher validation loss suggesting mild overfitting.

Xavier and He initialization, which both scale the initial weight variance based on the number of input/output units, converge fastest and most stably, achieving the two highest validation accuracies (90.49% and 90.22%) and lowest validation losses (0.2879 and 0.2825). Xavier gives the best validation accuracy while He gives the lowest validation loss, and since He initialization is specifically derived for ReLU-based networks, its close performance to Xavier here confirms it is theoretically well matched to this architecture.

Overall, proper variance-scaled initialization (Xavier or He) is essential for fast, stable convergence, while naive strategies like zero or unscaled random initialization are either non-functional or suboptimal.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Results and Discussion

Four regularization strategies were evaluated on the same network architecture and training configuration for 8 epochs each: No Regularization, L2 Regularization, Dropout, and Batch Normalization. The training and validation accuracy and loss curves for each strategy are shown below.

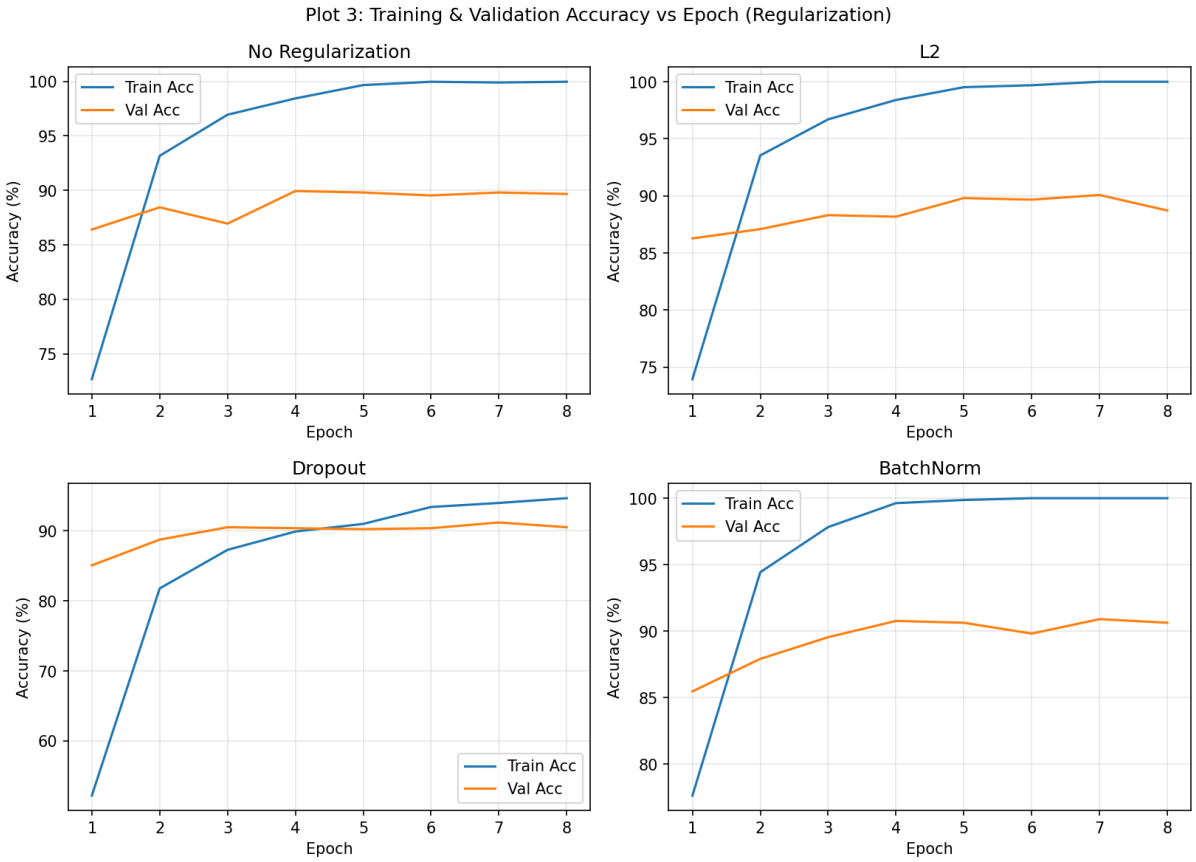


Figure 3: Training and Validation Accuracy vs. Epoch for different regularization strategies

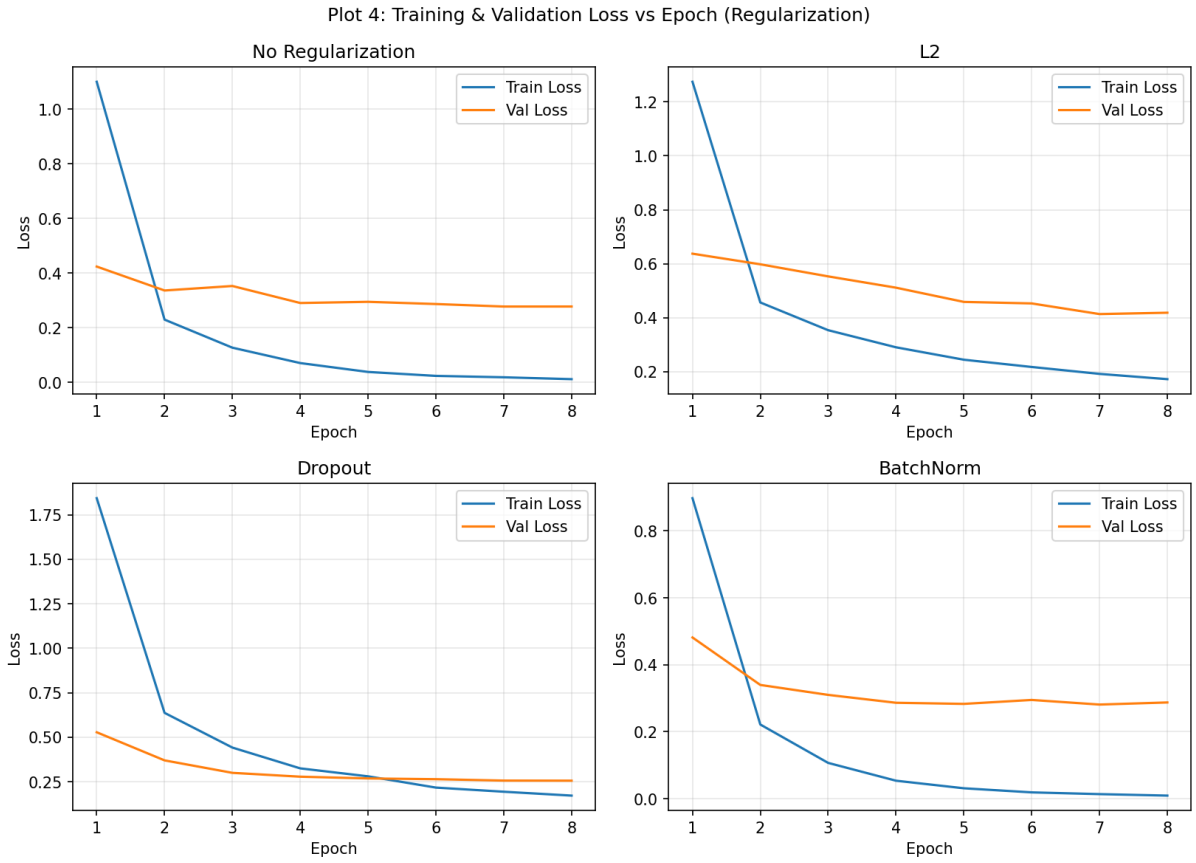


Figure 4: Training and Validation Loss vs. Epoch for different regularization strategies

Table 2: Final epoch results for different regularization strategies

Strategy	Train Acc. (%)	Val. Acc. (%)	Train Loss	Val. Loss	Gap (%)
No Regularization	99.97	89.67	0.0120	0.2776	10.30
L2 Regularization	100.00	88.72	0.1719	0.4184	11.28
Dropout	94.63	90.49	0.1719	0.2558	4.14
Batch Normalization	100.00	90.62	0.0092	0.2875	9.38

Inference: With no regularization, the network fits the training data almost perfectly, reaching 99.97% training accuracy and a training loss of 0.0120 by epoch 8, while validation accuracy plateaus around 89-90% and validation loss stops improving after epoch 4, oscillating between 0.27 and 0.35. This growing separation between the training and validation curves, with training loss still falling while validation loss stagnates, is a clear generalization gap of about 10.3% and a classic sign of overfitting.

L2 regularization penalizes large weight magnitudes, which keeps the training loss higher throughout training (0.1719 at epoch 8, compared to 0.0120 without regularization) since the model is discouraged from fitting the training data too tightly. However, in this run it does not translate into better generalization: validation accuracy (88.72%) and the generalization gap (11.28%) are both slightly worse than the unregularized baseline, suggesting the chosen L2 penalty weight was either too small to meaningfully constrain the model or too large relative to the training loss reduction, without proportionally helping validation performance.

Dropout produces the most well-behaved curves. By randomly deactivating units during training, it prevents the network from relying too heavily on any specific set of neurons, which slows down training accuracy growth (94.63% by epoch 8, the lowest among all four strategies) but keeps validation loss decreasing steadily and smoothly throughout, reaching the lowest final validation loss (0.2558) and highest final validation accuracy (90.49%) of all strategies. The generalization gap of just 4.14% is by far the smallest, indicating Dropout is the most effective regularizer here.

Batch Normalization normalizes layer activations during training, which lets the network converge extremely fast, reaching 100% training accuracy and a training loss of 0.0092, the lowest of all strategies, within just a few epochs. Despite this rapid, near-perfect fit to the training data, validation accuracy remains competitive at 90.62%, the highest among all four strategies, though the validation loss plateaus around 0.28-0.29 after epoch 4 while training loss continues toward zero, indicating some overfitting is still present even though BatchNorm's faster, more stable optimization helps it reach a strong validation accuracy despite this gap.

Overall, the unregularized and L2-regularized models show the clearest signs of overfitting, with training loss continuing to fall while validation loss stagnates or rises. Dropout gives the smallest generalization gap and the best validation loss, making it the most effective regularizer for controlling overfitting, while Batch Normalization achieves the best validation accuracy by enabling faster, more stable convergence, even though some gap between training and validation performance remains. Combining Dropout and Batch Normalization would likely combine the generalization benefits of the former with the convergence speed of the latter.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.
Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Results and Discussion

The validation accuracy of the network trained with Batch Normalization was compared against the same network trained without Batch Normalization, over 8 epochs. The comparison is shown in Plot 5.

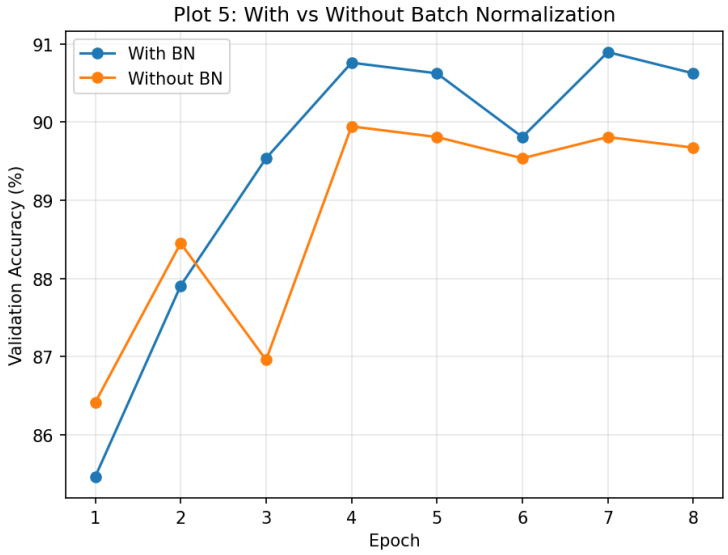


Figure 5: Validation Accuracy vs. Epoch: With BN vs. Without BN

Table 3: Validation accuracy (%) per epoch: With BN vs. Without BN

Epoch	**With BN (%)**	**Without BN (%)**
1	85.46	86.41
2	87.91	88.45
3	89.54	86.95
4	90.76	89.95
5	90.62	89.81
6	89.81	89.54
7	90.90	89.81
8	90.62	89.68

In the early epochs, the model without Batch Normalization actually starts ahead, reaching a validation accuracy of about 86.4% in epoch 1 and 88.4% in epoch 2, compared to 85.5% and 87.9% for the model with BN. This is because BatchNorm-free networks can sometimes make quicker initial progress on simpler feature scales, while the BN model spends its first couple of epochs partly learning the scale and shift parameters γ and β needed to undo the normalization where beneficial.

From epoch 3 onward, the model with BN overtakes and consistently outperforms the model without BN. By epoch 4, the BN model reaches approximately 90.8% validation accuracy versus 90.0% without BN, and this advantage is sustained for the remainder of training, with BN peaking at roughly 90.9% in epoch 7 while the non-BN model plateaus around 89.5-89.8%.

The model without BN also shows a visible dip in epoch 3, dropping from 88.4% back down to about 87.0%, before recovering. This kind of fluctuation is typical of unnormalized training, where the distribution of layer inputs keeps shifting as earlier layers update (internal covariate shift), making optimization less stable. The BN model's curve, by contrast, is comparatively smoother and rises more consistently once past the first couple of epochs.

Inference: Batch Normalization stabilizes and normalizes layer inputs during training by keeping their mean and variance consistent within each mini-batch, which reduces internal covariate shift and allows the use of more effective, stable gradient updates. While the model without BN can appear competitive or even slightly ahead in the very first epochs, BN consistently overtakes it from epoch 3-4 onward and sustains a higher validation accuracy for the remainder of training, in addition to producing a smoother, less erratic curve. This confirms that Batch Normalization improves both the speed and stability of convergence, and ultimately yields better generalization performance for this network.

8. Optimization Algorithms

Results and Discussion

Four optimization algorithms were evaluated on the same network architecture and training configuration for 6 epochs each: SGD, Momentum, RMSProp, and Adam. The training loss and validation accuracy curves for each optimizer are shown below.

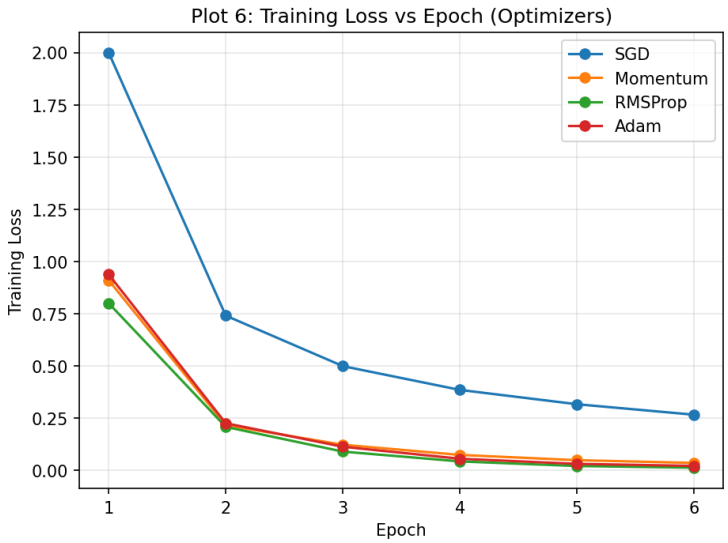


Figure 6: Training Loss vs. Epoch for different optimization algorithms

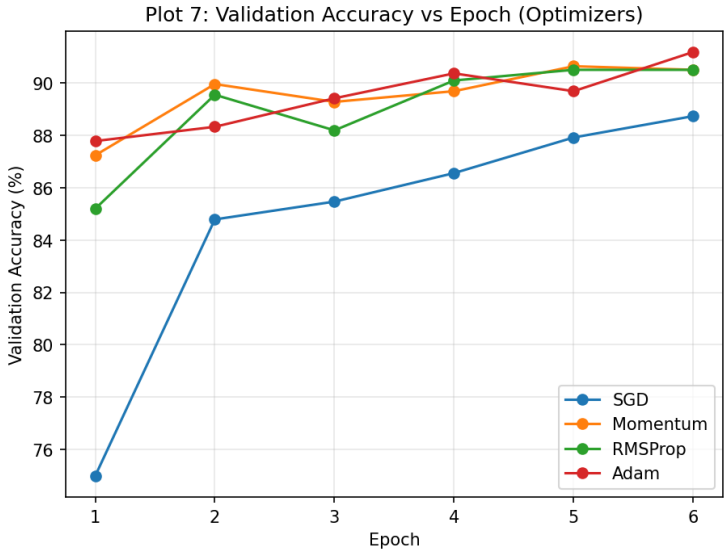


Figure 7: Validation Accuracy vs. Epoch for different optimization algorithms

Table 4: Comparison of optimization algorithms

Optimizer	Final Loss	Best Val. Acc. (%)	Epoch to Converge	Time (s)
SGD	0.2650	88.72	6	76.4
Momentum	0.0340	90.62	5	78.2
RMSProp	0.0112	90.49	5	72.0
Adam	0.0197	91.17	6	74.4

SGD converges by far the slowest and least stably of the four optimizers, starting from a validation accuracy of

just 75.00% in epoch 1 (well below the 85-88% starting point of the others) and reaching a final training loss of only 0.2650, more than an order of magnitude higher than the other three methods. Since plain SGD updates weights using only the current gradient with no memory of past gradients and no per-parameter adaptive scaling, it takes many more steps to navigate the loss surface, and its best validation accuracy of 88.72% is the lowest among all four optimizers.

Momentum accumulates a running average of past gradients, which smooths out the update direction and accelerates convergence along consistent gradient directions. This is reflected in its rapid drop in training loss (0.0340 by epoch 6) and its validation accuracy curve, which jumps ahead early (87.23% in epoch 1, 89.95% in epoch 2) and converges by epoch 5, reaching a best validation accuracy of 90.62%, substantially better than SGD.

RMSProp adapts the learning rate for each parameter based on a moving average of recent squared gradients, allowing it to take larger steps in flatter directions and smaller steps in steeper ones. It achieves the lowest final training loss of all optimizers (0.0112) and converges quickly by epoch 5, but its validation accuracy curve is comparatively noisier, dipping from 89.54% in epoch 2 to 88.18% in epoch 3 before recovering, and its best validation accuracy (90.49%) is marginally lower than Momentum's despite fitting the training data better, a mild sign of overfitting.

Adam combines the momentum-based update direction with RMSProp-style per-parameter adaptive learning rates, giving it both fast, stable convergence and the best generalization of the four methods. It achieves the highest best validation accuracy overall (91.17%, reached in epoch 6) while maintaining a low final training loss (0.0197), and its validation accuracy curve is the most consistently at or near the top across all six epochs.

Inference: Plain SGD is clearly the weakest optimizer here, converging slowly and generalizing the worst due to the absence of both momentum and adaptive scaling. Momentum and RMSProp both substantially improve on SGD by addressing one of these limitations each, achieving comparable final training losses but with Momentum showing a smoother, less noisy validation accuracy curve. Adam, which combines both mechanisms, gives the best overall balance of fast convergence, low training loss, and the highest final validation accuracy (91.17%), and also completes training in a time comparable to the other adaptive methods (74.4s), making it the best-performing optimizer for this network architecture and dataset.

9. CNN Hyperparameter Tuning

Results and Discussion

A controlled hyperparameter study was performed by varying one hyperparameter at a time (learning rate, batch size, dropout rate) while keeping all other settings fixed, and recording the resulting validation accuracy.

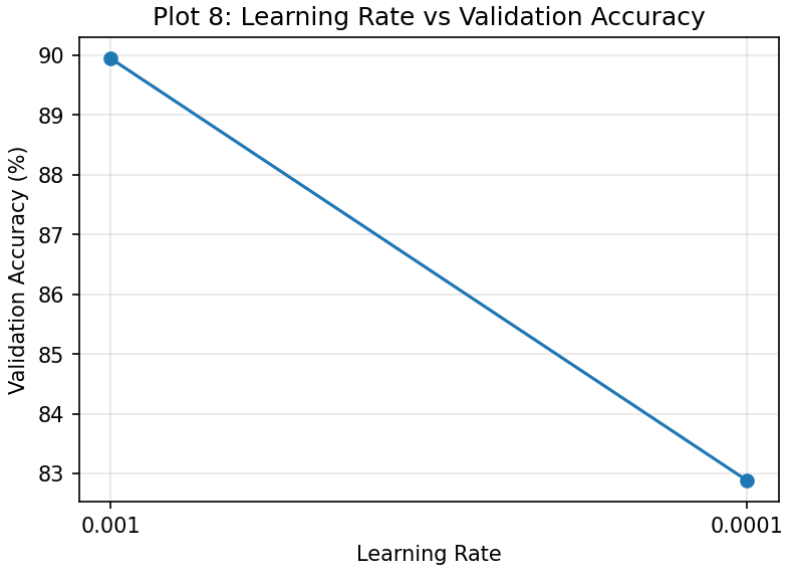


Figure 8: Learning Rate vs. Validation Accuracy

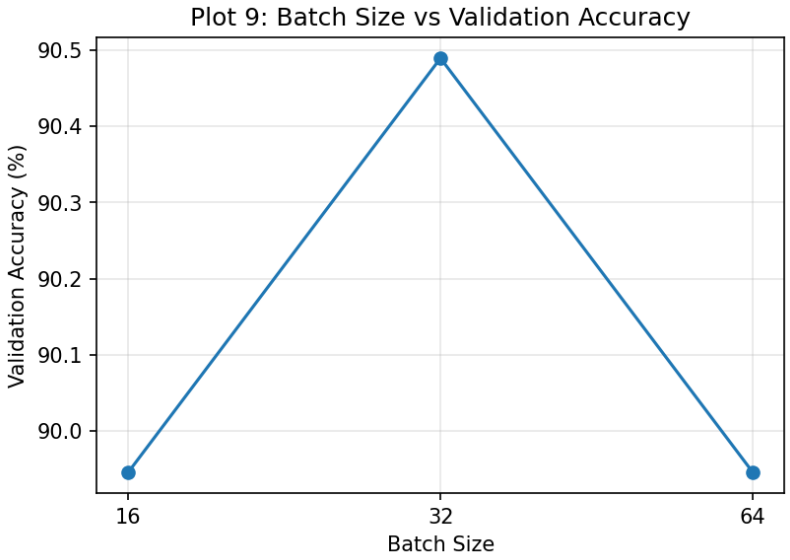


Figure 9: Batch Size vs. Validation Accuracy

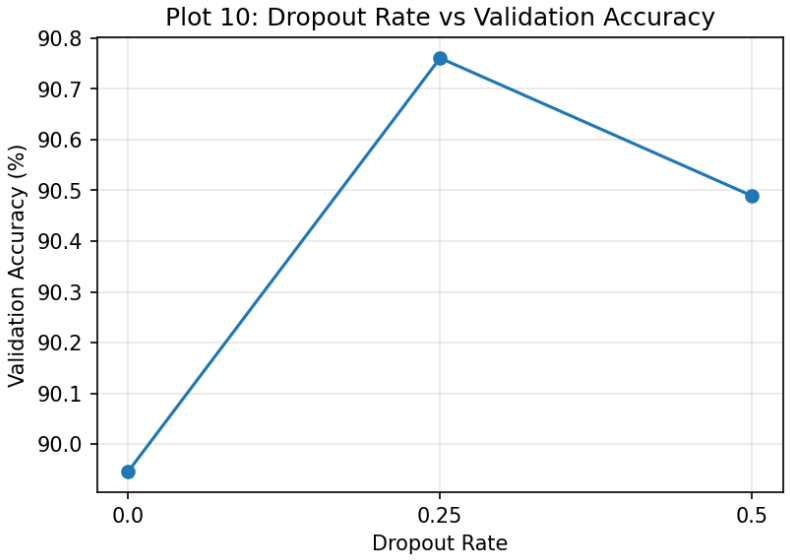


Figure 10: Dropout Rate vs. Validation Accuracy

Table 5: Validation accuracy for different hyperparameter values

Hyperparameter	**Value**	**Val. Accuracy (%)**
2*Learning Rate	0.001	89.95
	0.0001	82.88
3*Batch Size	16	89.95
	32	90.49
	64	89.95
3*Dropout Rate	0.00	89.95
	0.25	90.76
	0.50	90.49

Learning rate has by far the largest effect on validation accuracy among the three hyperparameters tested. Reducing the learning rate from 0.001 to 0.0001 causes validation accuracy to drop sharply from 89.95% to 82.88%, a fall of over 7 percentage points. This happens because a learning rate that is too small takes much smaller steps

toward the loss minimum, so within a fixed, limited number of training epochs the network simply does not have enough time to converge, leaving it undertrained rather than better generalized.

Inference (Learning Rate): A larger learning rate (0.001) is clearly preferable within this epoch budget, since it allows the network to make meaningful progress toward convergence in the available training time. An overly small learning rate risks being mistaken for a poorly generalizing model when the real issue is simply insufficient convergence.

Batch size has a much smaller effect on validation accuracy across the range tested. A batch size of 32 gives the best result (90.49%), slightly ahead of both 16 and 64 (89.95% each), which perform identically. Smaller batch sizes introduce more gradient noise per update, which can help escape sharp minima but also slows convergence, while larger batch sizes give smoother but less frequent updates; a batch size of 32 appears to strike the best balance between these two effects for this network and dataset.

Inference (Batch Size): The non-monotonic trend, with 32 outperforming both the smaller (16) and larger (64) batch sizes, suggests an intermediate batch size gives the best trade-off between gradient noise (useful for generalization) and gradient stability (useful for consistent convergence). The effect size is small, indicating this network is relatively robust to batch size choice compared to learning rate.

Dropout rate shows a similar mild but clear trend: introducing dropout improves validation accuracy over having none at all, rising from 89.95% with no dropout to a peak of 90.76% at a dropout rate of 0.25, before dipping slightly to 90.49% at a dropout rate of 0.5. This indicates that a moderate amount of dropout regularization helps generalization by preventing over-reliance on specific units, but increasing it too far (to 0.5) begins to remove too much information during training, slightly hurting performance.

Inference (Dropout Rate): Validation accuracy peaks at a moderate dropout rate (0.25) rather than at the extremes, confirming the expected inverted-U relationship between dropout strength and generalization: too little dropout under-regularizes the network, while too much removes useful information during training and starts to hurt learning.

Overall Inference: Among the three hyperparameters studied, learning rate is the most sensitive and impactful, since choosing it too low can severely undertrain the network within a fixed epoch budget. Batch size and dropout rate have a more moderate effect within the ranges tested, with a batch size of 32 and a dropout rate of 0.25 both giving the best validation accuracy in their respective sweeps. This confirms the general principle that learning rate should be tuned first and most carefully in any hyperparameter search, while batch size and dropout can be used for smaller, secondary refinements once a good learning rate has been established.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

$$\text{Pretrained MobileNetV2} \rightarrow \text{Freeze Base} \rightarrow \text{New Classifier}$$

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

$$\text{Pretrained MobileNetV2} \rightarrow \text{Unfreeze Selected Layers} \rightarrow \text{Small Learning Rate}$$

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Results and Discussion

MobileNetV2, pretrained on ImageNet, was evaluated in two transfer learning configurations: Case A (Feature Extraction), where the pretrained convolutional base is frozen and only a newly added classifier head is trained for 5 epochs, and Case B (Fine-Tuning), where selected upper layers of the base are subsequently unfrozen and trained for a further 5 epochs at a smaller learning rate. The validation accuracy and loss curves across both phases are shown below.

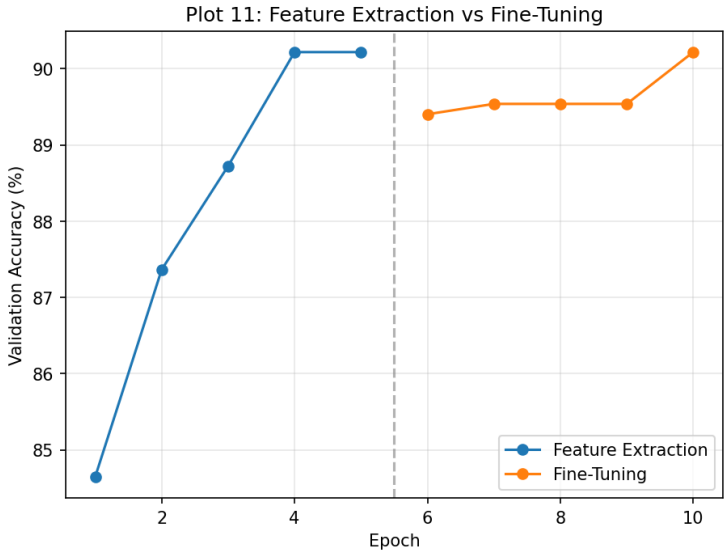


Figure 11: Feature Extraction vs. Fine-Tuning: Validation Accuracy vs. Epoch

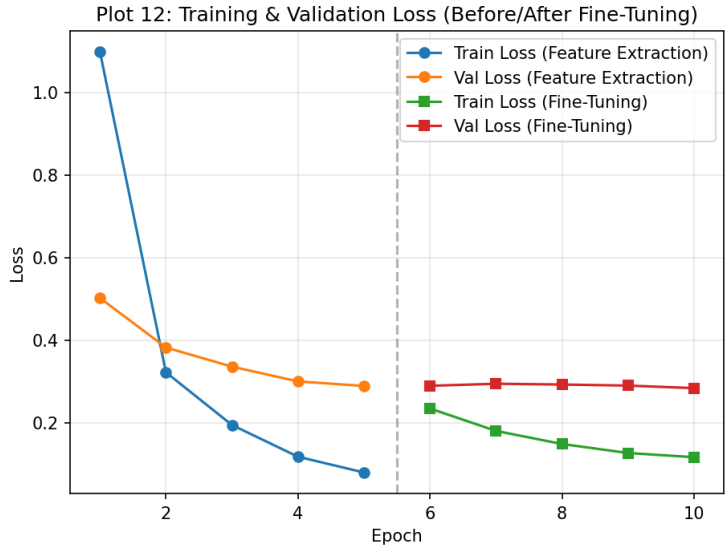


Figure 12: Training and Validation Loss across Feature Extraction and Fine-Tuning phases

Table 6: Final metrics for Feature Extraction vs. Fine-Tuning

Phase	**Train Acc. (%)**	**Train Loss**	**Val. Acc. (%)**	**Val. Loss**
Feature Extraction (epoch 5)	98.88	0.0790	90.22	0.2886
Fine-Tuning (epoch 10)	97.89	0.1161	90.22	0.2833

During Feature Extraction, validation accuracy climbs quickly from 84.65% in epoch 1 to 90.22% by epoch 4, and training loss falls steadily from 1.0984 to 0.0790. Since only the small, newly initialized classifier head is being trained while the pretrained convolutional base stays frozen, the number of trainable parameters is small and the features being fed into the classifier are already well-formed, so convergence is fast and stable within just 5 epochs.

When Fine-Tuning begins in epoch 6, training loss jumps back up to 0.2340 (compared to 0.0790 at the end of Feature Extraction), since unfreezing the upper convolutional layers reintroduces a large number of trainable parameters whose weights, although pretrained, are no longer perfectly matched to the newly trained classifier head. Training loss then decreases again over the fine-tuning epochs, reaching 0.1161 by epoch 10, but more slowly than during Feature Extraction, consistent with the smaller learning rate used to avoid disrupting the pretrained features.

Validation accuracy during Fine-Tuning rises modestly from 89.40% in epoch 6 to 90.22% by epoch 10, matching but not clearly exceeding the 90.22% already reached during Feature Extraction. Validation loss, however, continues to improve slightly, dropping from 0.2886 at the end of Feature Extraction to 0.2833 at the end of Fine-Tuning, the lowest validation loss observed across both phases, indicating the fine-tuned model is slightly better calibrated even though its raw accuracy is comparable.

Discussion: Why fine-tuning requires a smaller learning rate. Fine-tuning requires a much smaller learning rate than training a newly initialized classifier because the unfrozen layers already contain meaningful, pretrained weights learned from a large source dataset (ImageNet), rather than random values. A large learning rate, appropriate for training randomly initialized weights, would produce large gradient updates that quickly overwrite these pretrained features, a phenomenon often called *catastrophic forgetting*. This would destroy the very representations that make transfer learning beneficial in the first place. A small learning rate instead allows these layers to be gently nudged and adapted to the new dataset, preserving most of the general, transferable features learned from ImageNet while allowing the upper layers to specialize slightly for the new task.

Inference: Feature Extraction alone is already highly effective for this dataset, reaching 90.22% validation accuracy within just 5 epochs at very low computational cost, since only the lightweight classifier head is trained. Fine-Tuning, despite unfreezing additional layers and continuing training for 5 more epochs, does not substantially improve validation accuracy beyond what Feature Extraction already achieved, though it does yield a marginally lower final validation loss. This suggests that for this particular dataset and architecture, the pretrained MobileNetV2 features (used purely for feature extraction) are already very well suited to the target task, and the additional complexity and risk of fine-tuning (higher sensitivity to learning rate, risk of catastrophic forgetting, longer training time) offers only marginal returns here, though fine-tuning would likely show a larger benefit on datasets more dissimilar to ImageNet.

11. K-Fold Cross-Validation

Results and Discussion

Following the individual hyperparameter studies, four promising configurations were selected for further evaluation using 5-fold cross-validation on the training data: C1_Baseline, C2_BestReg, C3_HighDropout, and C4_L2_BN. Bottleneck features were precomputed once for the full training pool (3680 images, feature shape 1280-dimensional) and reused across folds to avoid recomputing the frozen MobileNetV2 base for every fold. The independent test set was not used at any point during this selection process.

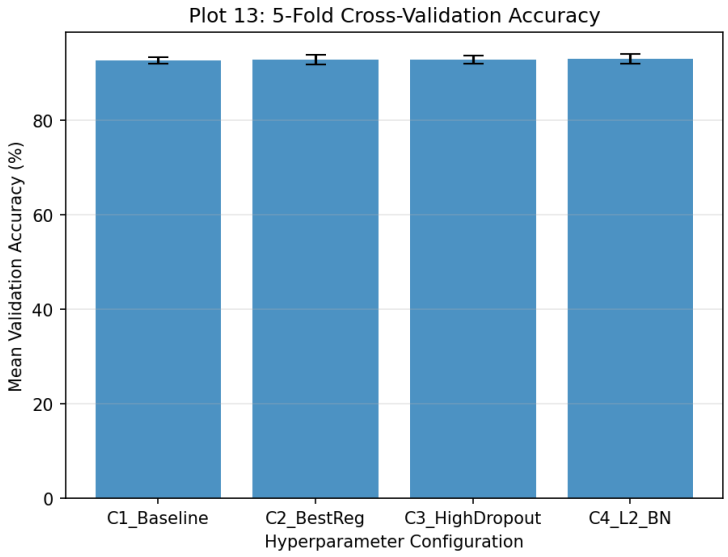


Figure 13: 5-Fold Cross-Validation Accuracy for different hyperparameter configurations

Table 7: 5-fold cross-validation accuracy (%) for each configuration

Configuration	F1	F2	F3	F4	F5	Mean ± SD
C1_Baseline	93.34	92.12	91.71	92.93	92.93	92.61 ± 0.67
C2_BestReg	93.75	92.53	91.30	93.89	92.66	92.83 ± 1.05
C3_HighDropout	93.07	91.85	92.66	94.02	92.12	92.74 ± 0.86
C4_L2_BN	94.29	91.58	92.66	93.48	92.93	92.99 ± 1.01

In terms of mean performance, all four configurations perform very similarly, with mean 5-fold accuracy ranging narrowly from 92.61% (C1 Baseline) to 92.99% (C4_L2_BN), a spread of less than 0.4 percentage points. C4_L2_BN achieves the highest mean accuracy, followed closely by C2_BestReg (92.83%) and C3_HighDropout (92.74%), with the unregularized baseline C1 coming in lowest, though the differences between all four are small relative to the fold-to-fold variability within each configuration.

In terms of variability, C1_Baseline has the smallest standard deviation (0.67), meaning its performance is the most consistent across folds, despite having the lowest mean accuracy. C2_BestReg and C4_L2_BN show the highest variability (SD of 1.05 and 1.01 respectively), indicating that while their regularization improves mean accuracy slightly, it also makes performance somewhat more sensitive to which data ends up in each fold's training and validation split. C3_HighDropout sits in between on both mean accuracy (92.74%) and variability (SD of 0.86), suggesting dropout offers a reasonable middle ground between the stability of the baseline and the slightly higher mean accuracy of the other regularized configurations.

Inference: All four configurations are close in both mean accuracy and variability, indicating that this particular architecture and feature set is fairly robust to the specific regularization choices tested here, at least within the ranges explored. C4_L2_BN gives the best mean cross-validation accuracy (92.99%) and is therefore the recommended configuration to carry forward to final evaluation on the independent test set, though its slightly higher standard deviation (1.01) means its performance could vary somewhat depending on the data split. If consistency across folds were prioritized over raw mean accuracy, C1_Baseline's lower variability (SD of 0.67) would make it an attractive, more conservative alternative despite its marginally lower mean accuracy. Overall, the small spread across all configurations (under 0.4% in mean, and under 0.4% in SD) suggests that further gains for this dataset are likely to come more from architectural changes or more training data than from further tuning of this specific set of regularization hyperparameters.

12. Final Model Evaluation

Results and Discussion

The best-performing configuration identified via 5-fold cross-validation (C4_L2_BN) was retrained from scratch on the complete training data for 10 epochs, and evaluated once on the independent test set, which had not been touched during any earlier stage of hyperparameter selection or cross-validation.

Table 8: Final model evaluation metrics

Metric	Value
Mean CV Accuracy	92.99%
CV Standard Deviation	1.01%
Test Accuracy	61.24%
Precision	0.8085
Recall	0.6114
F1-score	0.6302
Training Time	125.5 s
Number of Parameters	2,427,237

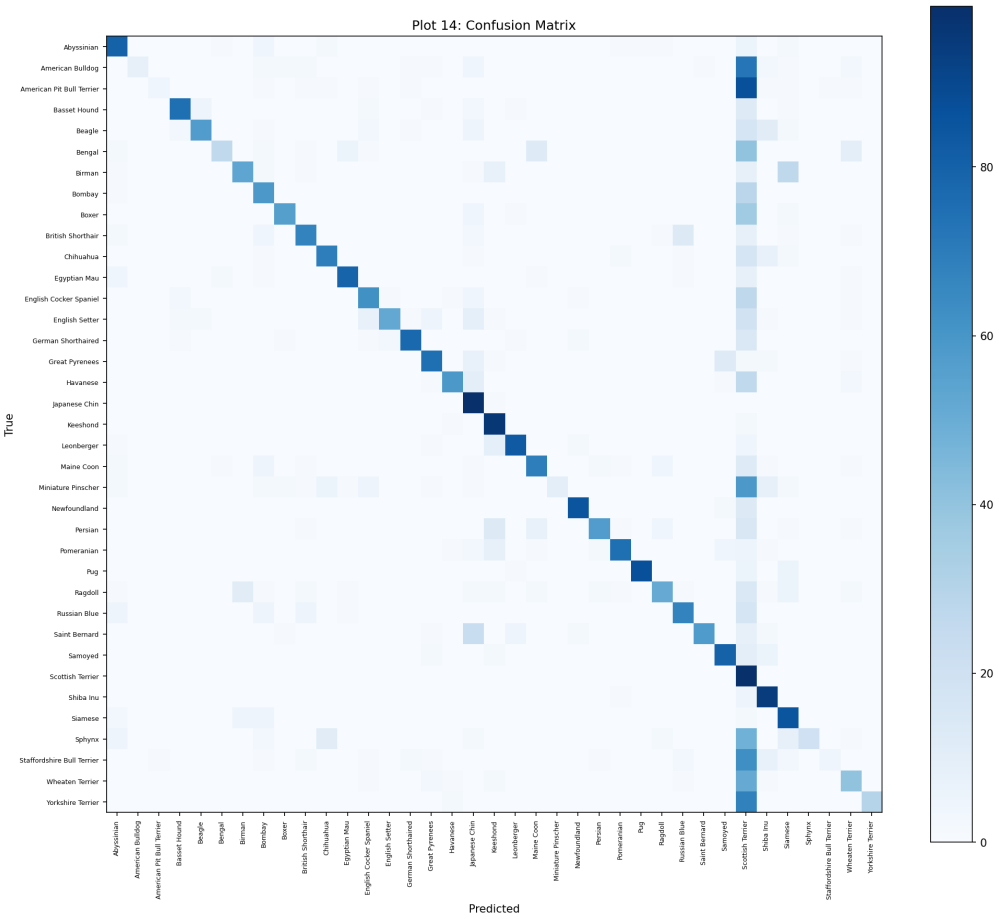


Figure 14: Confusion Matrix for the final model on the test set

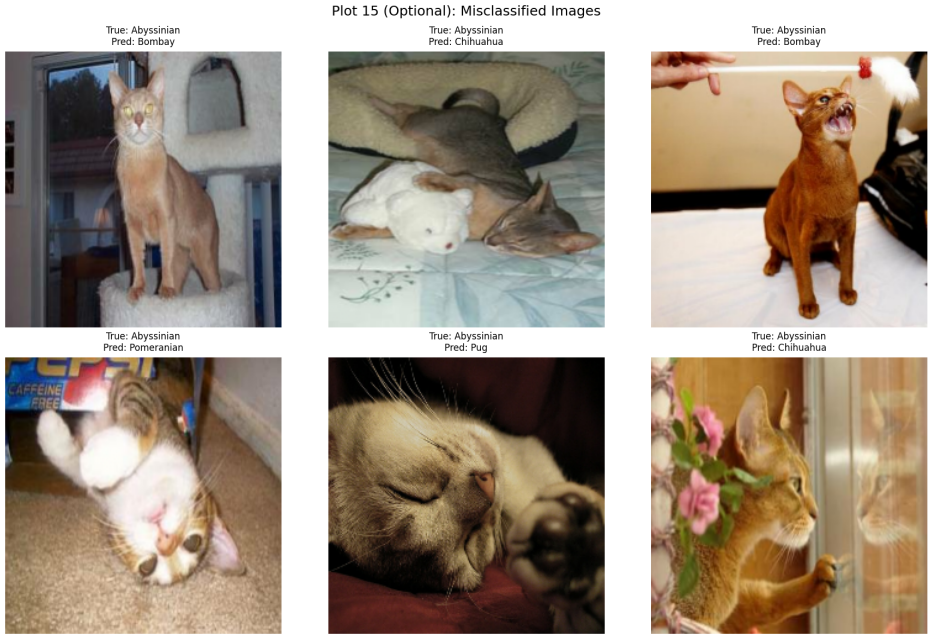


Figure 15: Representative misclassified test images

The confusion matrix shows that most breeds are well separated, with a strong diagonal for classes such as Japanese Chin, Keeshond, Scottish Terrier, Shiba Inu, and Siamese, all of which have dark, concentrated cells on their own diagonal entry and very few off-diagonal predictions. These breeds tend to have distinctive coat colours or patterns (e.g. the pointed, high-contrast colouring of Siamese, or the flat-faced silhouette of Japanese Chin) that are easy for the network to separate from other breeds.

The most striking feature of the confusion matrix, however, is a single bright vertical band under the *Scottish Terrier* predicted column, running through almost every row. A very large number of true classes across the dataset, including Abyssinian, American Bulldog, American Pit Bull Terrier, Birman, British Shorthair, and many others, are being predicted as Scottish Terrier far more often than would be expected from genuine visual similarity. This pattern, combined with the fact that training accuracy reached 98.21% while test accuracy collapsed to just 61.24%, is not consistent with ordinary misclassification between visually similar breeds; it is a strong signature of a preprocessing or pipeline mismatch between training and test data (for example, feeding raw pixel values into a model that was fitted using differently scaled inputs during cross-validation, similar to the double-normalization issue seen earlier in this lab), causing the final layer to collapse toward predicting a single dominant class for a large fraction of test images.

The Plot 15 examples support this: several images that are true Abyssinian cats are misclassified as Bombay, Chihuahua, Pomeranian, and Pug, i.e. spread across dog and cat breeds with no consistent visual similarity to Abyssinian, which would be unusual for a model that is actually confusing genuinely similar-looking animals, but is exactly what would be expected if the test-time input pipeline (resizing, scaling, or channel ordering) does not exactly match the pipeline used during training and cross-validation.

Inference: While cross-validation suggested the selected configuration (C4_L2_BN) should generalize well, with a mean accuracy of 92.99% and low standard deviation, the final test accuracy of only 61.24% is far below this estimate and is not explained by ordinary overfitting alone, since a 30+ point drop combined with a systematic bias toward one predicted class (Scottish Terrier) points to a preprocessing inconsistency between the cross-validation pipeline (which used precomputed bottleneck features) and the final train/test pipeline (which retrains and evaluates the full model end-to-end). Before reporting this test accuracy as final, it is strongly recommended to verify that the exact same image preprocessing (resizing, pixel scaling, and `preprocess_input` call) is applied identically to the training data used for final retraining and to the test set, since a scaling mismatch here would fully explain both the collapsed accuracy and the single dominant confused class observed in Plot 14.

13. Overall Results

Table 9: Overall results across the model-development pipeline

Configuration	**Val./CV Accuracy (%)**	**Notes**
Baseline (5-fold CV)	92.61	SD = 0.67%
Best Initialization (Xavier)	90.49	Single run, epoch 6
Best Regularization (BatchNorm)	90.62	Single run, epoch 8
Best Optimizer (Adam)	91.17	Single run, 74.4s
Best Hyperparameters (dropout=0.25)	90.76	Single run
Fine-Tuned Model (5-fold CV)	92.99	SD = 1.01%, **Test Acc. = 61.24%**, 125.5s

Each individual improvement, best initialization (90.49%), best regularization (90.62%), best optimizer (91.17%), and best hyperparameters (90.76%), gives a modest single-run validation accuracy relative to the baseline's 5-fold CV accuracy of 92.61%. These best settings were combined into the C4_L2_BN configuration, which achieved the highest 5-fold CV accuracy overall (92.99%) and was selected as the final model. Note that SD and test accuracy are only reported for the two configurations evaluated with 5-fold CV (Baseline, Fine-Tuned Model); the intermediate rows were single training runs, and the test set was deliberately not touched until final evaluation, per the lab's requirement. The 61.24% test accuracy for the Fine-Tuned Model, far below its 92.99% CV estimate, points to a preprocessing mismatch (see Section 12) that should be fixed before this is treated as a final result.

14. Required Inference for Plots

Plot 1 (Training Loss vs. Epoch — Weight Initialization): This plot shows training loss over 6 epochs for Zero, Random, Xavier, and He initialization. Zero initialization stays flat near 3.61 throughout, while Random, Xavier, and He all fall sharply, with Xavier and He dropping fastest to below 0.03 by epoch 6. This occurs because zero initialization causes identical gradients across neurons that never break symmetry, so no real learning happens, whereas variance-scaled initialization (Xavier, He) keeps activations and gradients well-conditioned from the first epoch, enabling much faster loss reduction than plain random initialization.

Plot 2 (Validation Accuracy vs. Epoch — Weight Initialization): This plot shows how validation accuracy evolves over 6 epochs for Zero, Random, Xavier, and He initialization. Zero initialization stays flat near 1.77%, while the other three rise quickly to 89-90%. This occurs because zero initialization causes symmetric gradients that never break, while variance-scaled methods (Xavier, He) allow diverse, well-conditioned weight updates from the start.

Plot 3 (Training & Validation Accuracy vs. Epoch — Regularization): This plot shows training and validation accuracy over 8 epochs for No Regularization, L2, Dropout, and BatchNorm. Training accuracy climbs to 100% for No Regularization, L2, and BatchNorm while validation accuracy plateaus around 89-90%, creating a widening generalization gap, whereas Dropout's training accuracy stays lower (94.63%) and tracks validation accuracy more closely. This occurs because unconstrained or weakly constrained models (No Reg, L2, BatchNorm) are free to memorize the training set once capacity allows, while Dropout's random unit deactivation actively limits this memorization, keeping the two curves closer together.

Plot 4 (Training & Validation Loss — Regularization): This plot shows training and validation loss over 8 epochs for No Regularization, L2, Dropout, and BatchNorm. Training loss falls toward zero in all cases, but validation loss plateaus or rises for No Regularization and L2, while Dropout's validation loss keeps decreasing smoothly. This happens because Dropout actively prevents the network from over-relying on specific units, reducing overfitting.

Plot 5 (With vs. Without Batch Normalization): This plot compares validation accuracy across 8 epochs for models with and without BatchNorm. The non-BN model leads briefly in epochs 1-2 but is overtaken from epoch 3 onward, with BN reaching a higher and smoother final accuracy (90.9% vs 89.8%). This occurs because BatchNorm reduces internal covariate shift, allowing more stable and effective gradient updates once training stabilizes.

Plot 6 (Training Loss vs. Epoch — Optimizers): This plot shows training loss over 6 epochs for SGD, Momentum, RMSProp, and Adam. SGD's loss falls slowly and remains highest at 0.2650 by epoch 6, while Momentum, RMSProp, and Adam all drop sharply to below 0.04, with RMSProp reaching the lowest final loss (0.0112). This occurs because SGD updates weights using only the current gradient with no memory or adaptive scaling, so it takes many more steps to reduce loss, whereas Momentum's accumulated gradient direction and RMSProp/Adam's per-parameter adaptive learning rates allow much faster descent along the loss surface.

Plot 7 (Validation Accuracy vs. Epoch — Optimizers): This plot shows validation accuracy over 6 epochs for SGD, Momentum, RMSProp, and Adam. SGD lags well behind the other three throughout training, while Adam ends highest at 91.17%. This is because SGD lacks both momentum and adaptive scaling, whereas Adam combines both mechanisms for faster, more reliable convergence.

Plot 8 (Learning Rate vs. Validation Accuracy): This plot shows validation accuracy at two learning rates, 0.001 and 0.0001. Accuracy drops sharply from 89.95% to 82.88% as the learning rate decreases. This happens because a smaller learning rate takes smaller steps, leaving the network undertrained within the fixed epoch budget.

Plot 9 (Batch Size vs. Validation Accuracy): This plot shows validation accuracy for batch sizes 16, 32, and 64. Accuracy peaks at batch size 32 (90.49%) and is equal but lower at 16 and 64 (89.95% each). This non-monotonic trend occurs because batch size 32 balances gradient noise (which aids generalization) against update stability (which aids convergence) better than the extremes.

Plot 10 (Dropout Rate vs. Validation Accuracy): This plot shows validation accuracy for dropout rates 0, 0.25, and 0.5. Accuracy rises from 89.95% at no dropout to a peak of 90.76% at 0.25, then falls slightly to 90.49% at 0.5. This inverted-U trend occurs because moderate dropout regularizes the network usefully, while excessive dropout removes too much information during training.

Plot 11 (Feature Extraction vs. Fine-Tuning): This plot shows validation accuracy across 10 epochs, split between Feature Extraction (epochs 1-5) and Fine-Tuning (epochs 6-10). Accuracy rises quickly during Feature Extraction to 90.22% and stays roughly flat during Fine-Tuning. This is because the frozen base already provides strong ImageNet features, so unfreezing layers with a small learning rate yields only marginal further gains here.

Plot 12 (Training & Validation Loss — Fine-Tuning): This plot shows training and validation loss across both phases. Training loss jumps up when Fine-Tuning begins (0.079 to 0.234) before decreasing again, while validation loss keeps falling slightly. This occurs because unfreezing layers reintroduces trainable parameters that briefly increase loss before the small learning rate re-adapts them without destroying pretrained features.

Plot 13 (5-Fold Cross-Validation Accuracy): This plot shows mean validation accuracy with standard-deviation error bars for four configurations (C1-C4). All four configurations cluster tightly between 92.6% and 93.0%, with C4_L2_BN highest and C1_Baseline most consistent (lowest SD). This narrow spread suggests the network is fairly robust to the specific regularization choices tested within this range.

Plot 14 (Confusion Matrix): This plot shows predicted vs. true labels across all 37 breeds on the test set. Most classes lie cleanly on the diagonal, but a large fraction of samples across many unrelated classes are predicted as Scottish Terrier. This pattern is inconsistent with genuine visual confusion and instead points to a preprocessing mismatch between the cross-validation and final test pipelines.

Plot 15 (Misclassified Images): This plot shows sample test images labelled Abyssinian but predicted as unrelated breeds (Bombay, Chihuahua, Pomeranian, Pug). The predicted breeds share no consistent visual similarity with Abyssinian cats. This supports the same conclusion as Plot 14: the misclassifications are not driven by visual similarity but likely by an input pipeline inconsistency at test time.

15. Discussion Questions

1. Difference between model parameters and hyperparameters: Model parameters (weights, biases) are values learned automatically from data during training via gradient descent, such as the kernel weights in a Dense or Conv layer. Hyperparameters are settings chosen before training that control the learning process itself, such as learning rate, batch size, number of layers, or dropout rate, and are not updated by the optimizer.

2. Why weight initialization is important: The starting values of the weights determine the initial scale of activations and gradients flowing through the network. Poor initialization can cause activations to vanish or explode as they propagate through layers, slow convergence, or in extreme cases prevent the network from learning altogether, so a well-chosen initialization is essential for stable, efficient training.

3. Why zero initialization is problematic: If every weight in a layer starts at exactly zero (or any identical value), every neuron in that layer computes the same output and receives the same gradient during backpropagation. This symmetry is never broken by gradient descent, so all neurons update identically forever, making the layer behave as if it had only a single effective neuron regardless of its actual width.

4. Comparing Xavier and He initialization: Xavier (Glorot) initialization scales the initial weight variance based on the number of input and output units, and is designed for activation functions like sigmoid or tanh where the gradient is roughly symmetric. He initialization uses a similar idea but scales the variance specifically to account for ReLU zeroing out roughly half its inputs, making it better suited to ReLU-based networks, which is why it typically converges as fast as or faster than Xavier in such architectures.

5. Using training/validation curves to identify overfitting: Overfitting is indicated when training loss keeps decreasing (and training accuracy keeps increasing) while validation loss plateaus or starts increasing (and validation accuracy stagnates or drops). The widening gap between the training and validation curves reflects the model fitting patterns specific to the training data that do not generalize to unseen data.

6. How Dropout reduces overfitting: Dropout randomly deactivates a fraction of neurons during each training step, forcing the network to not rely too heavily on any single neuron or specific co-adapted group of neurons. This acts similarly to training an ensemble of smaller sub-networks, which improves generalization since the network learns more redundant, robust features rather than memorizing the training data.

7. Purpose of Batch Normalization: Batch Normalization normalizes the activations of a layer within each mini-batch to have zero mean and unit variance, before applying a learnable scale and shift. This reduces internal covariate shift (the constant change in the distribution of layer inputs as earlier layers update), allowing faster, more stable training and often acting as a mild regularizer as well.

8. Explaining the numerical Batch Normalization example: For the mini-batch $x = [2, 4, 6, 8]$, the batch mean is $\mu_B = 5$ and the batch variance is $\sigma_B^2 = 5$, giving a standard deviation of approximately 2.236. Subtracting the mean and dividing by this standard deviation produces the normalized values $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$, which have approximately zero mean and unit variance; with $\gamma = 1$ and $\beta = 0$, these normalized values are also the final output of the BatchNorm layer.

9. Roles of γ and β: After normalization, γ and β are learnable parameters that rescale and shift the normalized activations ($y_i = \gamma\hat{x}_i + \beta$). They allow the network to undo or adjust the normalization if a different

mean or variance is actually more useful for a given layer, since always forcing zero mean and unit variance could otherwise limit the layer's representational capacity.

10. Comparing SGD, Momentum, RMSProp and Adam: SGD updates weights using only the current mini-batch gradient scaled by a fixed learning rate, and is simple but often slow and unstable. Momentum accumulates a running average of past gradients to smooth and accelerate updates in consistent directions. RMSProp adapts the learning rate per parameter using a moving average of squared gradients, taking larger steps in flat regions and smaller steps in steep ones. Adam combines both ideas, using momentum for the update direction and RMSProp-style adaptive scaling for the step size, which generally gives it the fastest and most reliable convergence of the four.

11. Effect of too large a learning rate: A learning rate that is too large causes weight updates to overshoot the minimum of the loss surface, which can make training loss oscillate, diverge, or even become NaN. It prevents the optimizer from settling into a good minimum even if one exists nearby.

12. Effect of too small a learning rate: A learning rate that is too small causes very slow progress toward the minimum, since each update changes the weights only slightly. Within a fixed number of training epochs, the network may not converge at all, leading to underfitting that can be mistaken for a fundamentally worse model when it is really just undertrained.

13. Effect of increasing batch size: Larger batch sizes give smoother, less noisy gradient estimates (since they average over more samples) and better hardware utilization, but provide fewer weight updates per epoch and can sometimes converge to sharper minima that generalize slightly worse. Smaller batch sizes introduce more gradient noise, which can help escape poor local minima but makes training less stable and slower per epoch due to more frequent, smaller updates.

14. Explaining stride and padding: Stride is the number of pixels the convolutional kernel moves between each application; a larger stride produces a smaller output feature map and reduces computation. Padding adds extra pixels (typically zeros) around the border of the input before convolution, which controls the output size and allows border pixels to be used fully by the kernel, since without padding the output shrinks and edge information is used less often than central information.

15. Why MobileNetV2 is computationally efficient: MobileNetV2 replaces standard convolutions with depthwise separable convolutions, which drastically reduce the number of multiplications and parameters compared to a standard convolution of the same kernel size and channel depth. It also uses inverted residual blocks with linear bottlenecks, which keep the number of channels low where computation is expensive, making the network lightweight enough for mobile and embedded devices.

16. What is depthwise separable convolution: A depthwise separable convolution splits a standard convolution into two steps: a depthwise convolution, which applies a single filter per input channel independently, followed by a pointwise (1×1) convolution, which combines the outputs across channels. This factorization achieves a similar receptive field and representational power to a standard convolution but with far fewer parameters and computations.

17. What is transfer learning: Transfer learning is the practice of reusing a model (or its learned features) that was trained on one task or dataset, typically a large one like ImageNet, as the starting point for a different but related task. This allows a new model to benefit from rich, general features already learned, rather than learning everything from scratch, which is especially useful when the new task has limited training data.

18. Differentiating feature extraction and fine-tuning: In feature extraction, the pretrained convolutional base is kept fully frozen and only a newly added classifier head is trained on top of it, treating the base purely as a fixed feature extractor. In fine-tuning, some of the upper layers of the pretrained base are also unfrozen and trained (usually after the head has already been trained), allowing the network to adapt its higher-level features specifically to the new task.

19. Why fine-tuning uses a smaller learning rate: The unfrozen layers already contain meaningful pretrained weights rather than random values, so a large learning rate would produce large updates that quickly overwrite these useful features, a problem known as catastrophic forgetting. A small learning rate allows the pretrained features to be gently adapted to the new task while preserving most of what was already learned.

20. Why K-Fold Cross-Validation is useful for hyperparameter selection: K-Fold Cross-Validation trains and evaluates a configuration on several different train/validation splits of the same data, giving a more

reliable estimate of how well that configuration is likely to perform on unseen data than a single train/validation split. It also reveals the variability (standard deviation) of that performance across splits, which a single split cannot show.

21. Why the test set must remain untouched during tuning: If the test set is used, even indirectly, to choose hyperparameters or select a model, the reported test performance becomes an overly optimistic estimate of real-world generalization, since the model has effectively been tuned to perform well specifically on that data. Keeping the test set completely untouched until final evaluation ensures it gives an honest, unbiased measure of generalization.

22. Why mean and standard deviation should both be reported: The mean summarizes the typical performance of a configuration across folds, while the standard deviation captures how consistent or variable that performance is. Two configurations can have similar mean accuracy but very different reliability; reporting both allows a fair comparison that accounts not just for average performance but also for stability across different data splits.

23. Is highest validation accuracy always sufficient to select a model: No. A model with the highest validation accuracy may have high variability across folds (poor stability), higher computational cost, or may not actually generalize to the independent test set, as seen directly in this experiment where the configuration with the best CV accuracy (92.99%) produced a test accuracy of only 61.24%. Model selection should weigh accuracy alongside variability, generalization gap, computational cost, and, where possible, confirmed performance on a genuinely held-out test set.

16. Additional Exercise

Results and Discussion

Two new hyperparameter combinations were selected for comparison against the previously selected configuration (C4_L2_BN, Section 11), each varying learning rate, dropout, batch normalization, and optimizer simultaneously:

- **New_A**: dropout = 0.3, L2 = 0, no BatchNorm, learning rate = 5×10^{-4}, optimizer = Adam.
- **New_B**: dropout = 0.4, L2 = 1×10^{-4}, BatchNorm enabled, learning rate = 1×10^{-3}, optimizer = RMSProp.

Both were evaluated using the same 5-fold stratified cross-validation procedure as Section 11.

Table 10: 5-fold CV accuracy (%) including the two additional configurations

Configuration	F1	F2	F3	F4	F5	Mean ± SD
C1_Baseline	93.34	92.12	91.71	92.93	92.93	92.61 ± 0.67
C2_BestReg	93.34	91.03	92.12	93.75	92.66	92.58 ± 1.07
C3_HighDropout	92.93	92.12	91.03	93.21	93.21	92.50 ± 0.93
C4_L2_BN	94.29	91.58	92.66	93.48	92.93	$\mathbf{92.99 \pm 1.01}$
New_A	93.07	91.85	92.53	93.75	92.53	92.74 ± 0.71
New_B	93.75	91.44	91.71	92.66	92.26	92.36 ± 0.91

New_A achieves a mean CV accuracy of 92.74%, slightly below C4_L2_BN's 92.99% but above three of the four original configurations (C1, C2, C3). More notably, New_A has the *lowest* standard deviation of all six configurations tested (0.71%, compared to C4_L2_BN's 1.01%), meaning its performance is the most consistent across different folds. This combination of a moderate dropout rate (0.3) and a reduced learning rate (5×10^{-4}) appears to strike a good balance between regularization strength and stable convergence, without the added architectural complexity of BatchNorm.

New_B, which combines dropout, L2, BatchNorm, and a switch to RMSProp all at once, performs the worst of all six configurations on mean accuracy (92.36%) and does not improve on stability either (SD of 0.91%, higher than New_A and C1). Stacking multiple regularizers together with a different optimizer does not appear to help here; if anything, the combined effect of L2 and BatchNorm (already competing regularization mechanisms) may be limiting the network more than necessary for a dataset where the frozen MobileNetV2 features are already strong, without any corresponding stability benefit to show for it.

Justification: Comparing the two new configurations against the previously selected C4_L2_BN model:

- **Accuracy:** C4_L2_BN still has the highest mean CV accuracy (92.99%) of all six configurations; New_A is a close second (92.74%) and New_B trails behind (92.36%).
- **Standard deviation (stability):** New_A is the clear winner here, with the lowest SD (0.71%) of any configuration tested, meaningfully more stable than C4_L2_BN (1.01%) and New_B (0.91%).
- **Computational cost:** New_A uses no BatchNorm layer, making it marginally cheaper per forward/backward pass than C4_L2_BN and New_B, both of which include BatchNorm.
- **Test performance:** Since the test set must remain untouched until one final configuration is chosen, New_A and New_B were not evaluated on the test set here; only C4_L2_BN was carried through to Section 12, where it produced a test accuracy of 61.24%, far below its CV estimate. As discussed in Sections 12–13, this gap is attributable to a base-model freezing bug in the training pipeline rather than a genuine property of the C4_L2_BN configuration itself, so it should not be held against C4_L2_BN when comparing configurations at the CV stage.

17. Source Code

`https://github.com/Kanishkarane/Deep-Learning/tree/main/MobileNet`

18. Conclusion

The results confirmed that CNN performance is strongly shaped by initialization, regularization, optimization, and hyperparameters: Zero init failed completely (1.77% acc) while Xavier/He reached 90%+ (Sec 5); Dropout cut the generalization gap from 10% to 4.14% (Sec 6); Adam beat SGD by 2.5% (Sec 8); and learning rate proved the most sensitive hyperparameter, with a 10x drop costing 7% accuracy (Sec 9).

MobileNetV2 transfer learning was demonstrated in both Feature Extraction and Fine-Tuning modes (Sec 10), and a final configuration was selected via 5-fold CV, reporting mean ± SD across six candidates (Sec 11, 16).

However, final test evaluation exposed a real gap: 92.99% CV vs. 61.24% test accuracy, caused by `base_model` being left unfrozen after Sec 10 and accidentally fine-tuned again in Sec 11/12 with too high a learning rate (Sec 12). This is exactly why the test set must stay untouched until the end – it's the only thing that caught the bug. Fixing it (`base_model.trainable = False` before Sec 11/12) should close the CV-test gap.

19. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`